

Contents

1	DD01: アカウント・ユーザー管理	1
1.1	0. 文書情報	1
1.2	1. 対象範囲	1
1.3	2. 収録ロジック・対応章	2
1.4	3. 詳細設計本文	2
1.4.1	3.1 利用者・権限詳細設計（参照のみ）	2
1.4.2	3.2 状態詳細設計（参照のみ）	2
1.4.3	3.3 画面詳細設計（参照のみ）	2
1.4.4	3.4 機能詳細設計（参照のみ）	2
1.4.5	3.5 実装モジュール構成	2
1.4.6	3.6 関連する横断設計	3
1.4.7	3.7 オーナー/メンバー/プロジェクト割当	3
1.4.8	3.8 退会フロー（参照のみ）	4
1.4.9	3.9 フロント URL ルーティング規約 (FR-325)	4
1.4.10	3.10 SCR-026 OnboardingChecklist 完了判定ロジック (FR-326)	5
1.4.11	3.11 契約共通項目の Used by Projects 集計 (FR-327)	5
1.5	4. 関連設計	5
1.6	5. テスト観点	6
1.6.1	5.1 結合テスト例（ロックアウト）	6
1.6.2	5.2 その他観点	6
1.7	6. 未確定事項・確認事項	7

1 DD01: アカウント・ユーザー管理

1.1 0. 文書情報

項目	内容
文書名	DD01: アカウント・ユーザー管理
詳細設計 ID	DD01
対象システム	FAQ AI ウィジェット SaaS/メインシステム
関連機能 ID	FR-001～022（新規登録/メール確認/パスワード再設定/ログイン/ロックアウト/管理者ユーザー管理/再認証/利用規約再同意）/FR-325～327(URL 規約/OnboardingChecklist/Used by Projects)
作成日	2026-05-17
版数	v1.1
ステータス	承認済

1.2 1. 対象範囲

種別	ID	名称
機能	FR-001～009	新規登録/メール確認/パスワード再設定/ログイン/ログアウト/再認証/ロックアウト
機能	FR-015～022	管理者ユーザー登録・停止・削除・有効化/メール再送/退会
機能	FR-011	利用規約再同意フロー
画面	SCR-001	ログイン
画面	SCR-002	新規登録
画面	SCR-003	パスワード再設定
画面	SCR-017	管理者ユーザー一覧
画面	SCR-026	契約ホーム (OnboardingChecklist + Used by Projects、§ 3.10/ § 3.11)
機能	FR-325～327	URL ルーティング 4 系列規約/初回セットアップガイド/契約共通項目の影響範囲表示

種別	ID	名称
API	/auth/* /admin-users/* /me/email-verification/* /me/terms* /me/onboarding-status	認証認可・管理者ユーザー操作・規約同意・オンボーディング状態
テーブル	users contract_owners project_users sessions access_tokens	アカウント基盤(認証情報 = users、オーナー専有属性 = contract_owners、プロジェクト別ロール admin / member = project_users.role)

1.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 4	利用者・権限詳細設計	権限設計書 / 認証認可設計書への参照（正本は基本設計）
§ 6 SCR-001~003,017	画面詳細設計	ログイン / 新規登録 / パスワード再設定 / 管理者ユーザー一覧（正本は基本設計）
§ 7	機能詳細設計（アカウント・ユーザー機能）	認証フロー / 管理者ユーザー管理 / 規約同意（正本は基本設計）
§ 3.3.1	worker-main-api モジュール構成	routes/auth.ts routes/admin-users.ts routes/email-verification.ts routes/terms.ts

1.4 3. 詳細設計本文

1.4.1 3.1 利用者・権限詳細設計（参照のみ）

ユーザー種別、3 ロール（オーナー / プロジェクト管理者 / メンバー）、画面別権限マトリクス、認証フロー、Principal 構築は [../02_基本設計/04_権限設計.md](#) および [../02_基本設計/08_認証・認可設計.md](#) を正本とする。

利用者側アカウントは **オーナー**（契約あたり 1 ユーザー固定、全権、MVP では譲渡不可、contract_owners 行を持つユーザー = オーナー判定、プロジェクト作成時に **project_users(role='admin', valid=1)** 行を自動取得する明示的なプロジェクト管理者保持者となる。認可上は isOwner=true(contract_owners 行存在)bypass を優先）と **メンバー**（0..N、所属契約は project_users.contract_owner_user_id 経由で contract_owners.user_id に紐付き、プロジェクトごとに project_users.role = admin(プロジェクト管理者) / member(メンバー) のいずれかを持つ）に分かれる。オーナー専有機能（プロジェクト作成・編集・削除、課金、退会、規約再同意）はプロジェクト別ロールでは付与できない。**メンバーアカウント削除はプロジェクト単位の SCR-017-M1「アカウントを削除」(DELETE /projects/{id}/members/{userId}) で行う論理削除 (users.valid=0)**（基本設計 FR-019 / FR-019a / FR-019b 参照）。論理削除データは updated_at 基準で 90 日経過後に物理削除バッチ（DD14）で物理削除される。論理削除済 users.valid=0 行はログイン拒否されるため割当 0 件状態は構造的に発生しない。プロジェクト削除起因で valid=1 割当が 0 件となるメンバーは同一トランザクションで users.valid=0 に論理削除する。

1.4.2 3.2 状態詳細設計（参照のみ）

契約状態（contract_owners.contract_status: active / suspended / deleted_pending / deleted）の全状態遷移は [../02_基本設計/03_テーブル設計.md](#) § 8 を正本とする。

1.4.3 3.3 画面詳細設計（参照のみ）

SCR-001 ログイン / SCR-002 新規登録 / SCR-003 パスワード再設定 / SCR-017 管理者ユーザー一覧の表示・入力・操作・バリデーション・モーダル・状態別表示・メッセージ ID は [../02_基本設計/01_画面設計.md](#) および [../02_基本設計/06_メッセージ一覧.md](#) を正本とする。

1.4.4 3.4 機能詳細設計（参照のみ）

機能別のロジック・データ・API 連動は [../02_基本設計/02_API 設計.md](#) および [../02_基本設計/03_テーブル設計.md](#) を正本とする。本書では実装モジュール構成のみを扱う。

1.4.5 3.5 実装モジュール構成

app/workers/main-api/src/ 配下に以下のルート / ハンドラ / リポジトリを配置する。

```
src/
├─ routes/
```

```

|   |— auth.ts                # POST /auth/login / POST /auth/logout / POST /auth/password-
reset
|   |— admin-users.ts         # GET/POST /admin-users/* (管理者ユーザー CRUD)
|   |— email-verification.ts # POST /me/email-verification/{verify,resend}
|   |— terms.ts               # GET /me/terms / POST /me/terms/agree
|   |— handlers/              # 上記 routes と 1:1
|   |— domain/
|   |   |— contract-status.ts # contract_owners.contract_status 遷移ガード (active → suspended → del
|   |   |— repository/
|   |   |   |— users.ts        # users(認証情報) CRUD
|   |   |   |— contract-owners.ts # contract_owners(オーナー専有属性 = 契約状態 / data_deletion_mode) CR
|   |   |   |— project-users.ts # project_users(プロジェクト割当 + role 列含む) CRUD
|   |   |   |— sessions.ts
|   |   |   |— access-tokens.ts
|   |   |— middleware/
|   |   |   |— authenticate.ts # Cookie + CSRF
|   |   |   |— require-active-contract.ts
|   |   |   |— require-terms-agreement.ts
|   |   |   |— require-reauth.ts # 重要操作再認証
|   |   |   |— csrf.ts
|   |   |— lib/
|   |   |   |— argon2id.ts      # hashPassword / verifyPassword
|   |   |   |— token.ts        # email_verify / password_reset / activation トークン

```

1.4.6 3.6 関連する横断設計

1.4.6.1 3.6.1 トークン発行・検証 メール確認・パスワードリセット・管理者ユーザー有効化 (activation) の各トークンは [DD08_トークン発行・検証.md](#) の HMAC-SHA256 保存方式を利用する。一回限りトークンは `consumeToken()` を検証直後に必ず呼ぶ。

1.4.6.2 3.6.2 認可ヘルパ `requireTenant / requireProject / requireRole` の各ヘルパは [DD09_認可ヘルパ.md](#) を利用する。

1.4.6.3 3.6.3 監査ログ書込 `auth.signup / auth.login_success / auth.login_failure / auth.logout / auth.password_reset / auth.re_auth.success / auth.ip_blocked / auth.terms_agree` などの `action` コードはすべて [DD10_監査ログ書込・完全性検証.md](#) の `writeAudit()` で書込み、`retention_class=general` とする。

1.4.6.4 3.6.4 暗号化・鍵管理 `email / phone` などの PII はオーナー派生 で AES-256-GCM 暗号化し、検索用に HMAC-SHA256 列 (`email_hmac` 等) を併設する。詳細は [DD11_暗号化・管理.md](#) を参照。

1.4.7 3.7 オーナー / メンバー / プロジェクト割当

- ・オーナー判定 = `contract_owners` 行 (`valid=1`) を持つユーザー。 `contract_owners.user_id` が当該オーナーの `users.id` を直接参照する (`is_owner` カラムも `owner_account_id` 自己参照も持たない)
- ・メンバーは `contract_owners` 行を持たず、 `project_users.contract_owner_user_id` 経由で所属契約に紐付く
- ・メンバーの権限は `project_users.role(admin / member)` とプロジェクト割当の組合せで管理 (個別権限フラグ方式は MVP で廃止)
- ・オーナーもプロジェクト作成時に `project_users(role='admin', valid=1)` 行を自動取得し、明示的なプロジェクト管理者として行を保持する。認可フロー先頭の `isOwner=true` `bypass(contract_owners` 行存在で判定) は維持し、 `project_users` 行は通知宛先解決と画面表示の事実情報として参照する
- ・新規オーナー登録時のトランザクション: `INSERT INTO users(...)` → `INSERT INTO contract_owners(user_id, contract_status='active', valid=1, ...)` を同一 BEGIN IMMEDIATE で実行 (`users.id = contract_owners.user_id = contractOwnerUserId`、物理値は同一 ULID)
- ・共有概念は [../共有/共有概念.md](#) を正本とする

1.4.8 3.8 退会フロー (参照のみ)

- ・オーナーが退会要求 → UPDATE contract_owners SET contract_status='deleted_pending' + withdrawal_requests レコード作成 + scheduled_deletion_at (30 日後)
- ・30 日経過後、§ 14.1.9 定期削除 cron が UPDATE contract_owners SET contract_status='deleted', valid=0 に更新し、後続バッチで users.valid=0 を揃える
- ・詳細は [DD14_バッチ・非同期処理.md](#) を参照

1.4.9 3.9 フロント URL ルーティング規約 (FR-325)

利用者向け管理コンソールの SPA は **スコープ別 4 系列** のルートツリーで構成する。スコープ判定はフロント router の beforeEach ガードと API ミドルウェア authenticate.ts + requireTenant の二重化で行う。

/auth/*	— 認証フロー(未認証可)
/owner/*	— 契約 WS(オーナー専有 = `contract_owners` 行存在)
/projects/:projectId/*	— プロジェクト WS(`project_users` 保持者)
/account/*	— 共通領域(認証済全員)
/eu/*	— エンドユーザー(認証コンソール外、トークン認証)

1.4.9.1 3.9.1 フロント router のガード (疑似コード)

```
router.beforeEach(async (to) => {
  const scope = parseScope(to.path) // 'auth' | 'owner' | 'projects' | 'account' | 'eu'
  const me = await ensureSession() // 未認証なら null

  if (scope === 'auth') {
    if (me) return resolveDefaultWorkspace(me) // 認証済は既定 WS へリダイレクト (§4.4.7)
    return true
  }

  if (!me) return '/auth/login'

  if (scope === 'owner' && !me.isOwner) {
    return resolveDefaultWorkspace(me) // 403 同等の扱い、UI 上は権限不足トーストを表示
  }

  if (scope === 'projects') {
    const projectId = to.params.projectId
    const projectUser = me.projectUsers.find(pu => pu.project_id === projectId)
    if (!me.isOwner && !projectUser) {
      // ユーザー削除は論理削除 (users.valid=0、project_users も valid=0 連動)。「me.projectUsers」は
      return resolveDefaultWorkspace(me)
    }
  }

  return true
})
```

1.4.9.2 3.9.2 既定ワークスペース決定 (§4.4.7 と同期) resolveDefaultWorkspace(me) は次の順で評価する:

1. localStorage の lastWorkspace が現在の project_users で有効ならその WS のホーム
2. オーナーかつ未保存なら /owner(SCR-026)
3. メンバーで project_users 行が1件なら /projects/{onlyRow.project_id}(SCR-015)
4. メンバーで project_users 行が複数なら最新 last_accessed_at のプロジェクト

ユーザー削除はユーザーレベル論理削除 (users.valid=0) のため、現存するメンバー (users.valid=1) は project_users.valid=1 を1件以上満たし、「project_users 行が0件のケース」は構造的に発生しない。データ不整合等で WHERE valid=1 条件下で projectUsers.length === 0 を検出した場合のフェイルセーフは Sentry へ重大度 high を通知し /auth/login へ強制ログアウトする (運営者 IF #2 強制ログアウト発火を期待)。

認証ミドルウェアの追加チェック: 認証成功直後に users.valid=1 を確認し、valid=0 の場合は 401 でログイン拒否 (セッション失効扱い)。これにより論理削除済みユーザーのセッションが残存しても再アクセスを防止

する。オーナーの場合は加えて `contract_owners.valid=1 AND contract_status NOT IN ('deleted')` も検証 (`deleted_pending` は 30 日間の猶予期間中は許容)。

1.4.10 3.10 SCR-026 OnboardingChecklist 完了判定ロジック (FR-326)

オーナーが SCR-026 契約ホームに到達したとき、画面ロード時に以下のクエリを **1 リクエストの集約 API**(`GET /me/onboarding-status`) で取得し、3 ステップそれぞれの完了状態を返す。リアルタイム購読はしない (MVP)。

ステップ	完了条件 (SQL レベル)	返却フィールド
1. プロジェクト作成	<code>SELECT EXISTS(SELECT 1 FROM projects WHERE contract_owner_user_id = :contractOwnerUserId AND deleted_at IS NULL)</code>	<code>step_project_created:</code> boolean
2. メンバー招待	<code>SELECT EXISTS(SELECT 1 FROM project_users pu JOIN projects p ON pu.project_id = p.id WHERE p.contract_owner_user_id = :contractOwnerUserId AND pu.user_id != :contractOwnerUserId)</code>	<code>step_member_invited:</code> boolean
3. ウィジェット公開	<code>SELECT EXISTS(SELECT 1 FROM widget_keys wk JOIN projects p ON wk.project_id = p.id WHERE p.contract_owner_user_id = :contractOwnerUserId AND wk.published_at IS NOT NULL)</code>	<code>step_widget_published:</code> boolean

1.4.10.1 3.10.1 表示制御

- 3 ステップ全て `true` の場合、フロントは OnboardingChecklist セクション全体を非表示 (画面再ロードのたびに再判定し、いったん全完了になった後でも条件が崩れれば再表示する。MVP では「完了状態を `localStorage` に固定しない」)
- ステップ 1 が未完了の場合、ステップ 2/3 の CTA をグレースアウト + tooltip(**MSG-SCR-026-T00LTIP-001**)
- 各ステップ CTA の遷移先 URL は § 3.9 の規約に従う:
 - ステップ 1: `/owner/projects/new`(SCR-010-M1 を新規モードで開く)
 - ステップ 2: 最新作成プロジェクトの `/projects/{projectId}/members` → 招待モーダル (SCR-017-M1。SCR-017 はプロジェクト WS 単一視点)
 - ステップ 3: 最新作成プロジェクトの `/projects/{projectId}/widget`(SCR-014)

1.4.10.2 3.10.2 実装モジュール配置

- ハンドラ: `app/workers/main-api/src/handlers/onboarding-status.ts`(新規)
- ルート: `app/workers/main-api/src/routes/me.ts` に `GET /me/onboarding-status` を追加
- 認可: `requireTenant + requireOwner(contract_owners 行存在で判定)`。非オーナーは **403 E-AUTHZ-OWNER-ONLY**
- レスポンスキャッシュ: なし (画面ロードごとに新規評価。クエリは 3 件の `EXISTS` のみで軽量)

1.4.11 3.11 契約共通項目の Used by Projects 集計 (FR-327)

契約共通バッジ付き項目 (MVP では月次予算が主要対象) の **Used by Projects: N** 補助表示用に、各項目ごとの参照プロジェクト件数を返す API を提供する。

- 月次予算: `SELECT COUNT(*) FROM projects WHERE contract_owner_user_id = :contractOwnerUserId AND deleted_at IS NULL`(契約配下の全プロジェクト数。月次予算は契約単位で適用されるため、全プロジェクトが利用先となる)
- 集約レスポンスは `GET /me/onboarding-status` と同一 API にまとめて返却 (`shared_assets_usage` フィールド) し、SCR-026 の 1 ロードで全データを揃える

N=0 の場合のラベル切替は基本設計 [01_画面設計.md § 4.4.4](#) のメッセージ ID 体系 (**MSG-SCR-026-LBL-005 / LBL-006**) に従う。

1.5 4. 関連設計

種別	参照先
要件	<code>../01_要件定義/index.md</code>
基本設計	<code>../02_基本設計/index.md</code>
権限設計	<code>../02_基本設計/04_権限設計.md</code>

種別	参照先
認証・認可設計	../02_基本設計/08_認証・認可設計.md
共有概念	../共有/共有概念.md
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md
関連 DD	DD08_トークン発行・検証.md / DD09_認可ヘルパ.md / DD10_監査ログ書込・完全性検証.md / DD11_暗号化・管理.md

1.6 5. テスト観点

AC ID	テスト ID	テスト方式	テストファイル
AC-001	e2e-signup-001	E2E	apps/admin/e2e/auth/signup.spec.ts
AC-002	e2e-email-verify-001	E2E	apps/admin/e2e/auth/email-verify.spec.ts
AC-003	e2e-pwreset-001	E2E	apps/admin/e2e/auth/password-reset.spec.ts
AC-004	e2e-login-001 / e2e-logout-001	E2E	apps/admin/e2e/auth/login.spec.ts / ../logout.spec.ts
AC-005	it-reauth-001	Integration	workers/main-api/test/integration/auth/reauth.test.ts
AC-007	it-lockout-001	Integration	workers/main-api/test/integration/auth/lockout.test.ts
AC-008	it-session-expire-001 / it-session-expire-002	Integration	workers/main-api/test/integration/auth/session-expire.test.ts
AC-009	e2e-admin-user-activation-001	E2E	apps/main/e2e/admin-users.spec.ts
AC-016	e2e-withdrawal-001	E2E	apps/admin/e2e/account/withdrawal.spec.ts
AC-027	e2e-terms-modal-001	E2E	apps/admin/e2e/terms/reaccept.spec.ts
AC-046	it-force-logout-001	Integration	workers/main-api/test/integration/owner/force-logout.test.ts

1.6.1 5.1 結合テスト例（ロックアウト）

```
// tests/integration/auth/login.test.ts
describe('POST /api/v1/auth/login', () => {
  it('正しい資格情報でログイン成功', async () => {
    await seedAccount({ email: 'admin@example.com', password: 'P@ss12345678', role: 'admin' });
    const res = await (await worker).fetch('http://localhost/api/v1/auth/login', {
      method: 'POST', body: JSON.stringify({ email: '...', password: '...' }),
    });
    expect(res.status).toBe(200);
    expect(res.headers.get('Set-Cookie')).toContain('session=');
  });
  it('5 回失敗でロックアウト', async () => {
    for (let i = 0; i < 5; i++) {
      await (await worker).fetch('...', { method: 'POST', body: JSON.stringify({ ..., password: '...' })
    }
    const res = await (await worker).fetch('...', { method: 'POST', body: JSON.stringify({ ..., password: '...' })
    expect(res.status).toBe(423); // LOCKED_OUT
  });
});
```

1.6.2 5.2 その他観点

観点	内容
単体テスト	contract-status.ts 遷移ガード（全 4 状態 × 全遷移を網羅）
異常系	ロックアウト / 期限切れトークン / リプレイトークン / 規約未同意時の API 拒否
境界値	パスワード長 12 文字 / 失敗回数 5 回 / セッション TTL 12h
権限	オーナー専有機能（課金 / 退会 / 規約再同意）にメンバーアクセス時 403
性能	POST /api/v1/auth/login p95 < 800ms（Argon2id verify 支配）

1.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済